EUROPEAN JOURNAL OF SCIENCE AND MODERN TECHNOLOGIES
ISSN 3083-6565
open access
peer reviewed

Continuous Data Curation and Valuation for Long-Term Machine Learning Model Health: A Comprehensive Review

Mehedi Hasan[1], Shayma Islam Shifa[2], Kashif Niaz[2], Md Mahedi Hasan Shuvo[2]
[1]*Information and Communication Engineering, Xi'an Jiaotong University, China*
[2]*Computer Science and Technology, Changsha University of Science and Technology, China*

Abstract

The long-term efficacy of Machine Learning (ML) models hinges on the quality of the training data used within ML systems. The real-world application of ML systems involves changes in contextual data over time, leading to drift. This drift over time leads to a reduction in the model's accuracy, resilience, and overall reliability. This phenomenon has been named the "AI ageing". The purpose of this review is to illustrate key concepts, techniques, and methodologies developed to address the challenges of continuous data curation and data valuation. It consolidates research within automated data cleaning, drift detection, data valuation, active learning, and MLOps to provide a cohesive perspective on data-centric concerns in contemporary ML systems. The review elaborates on the definitions and metrics of model health, continual data curation, and data valuation, as well as the detection and response to data condition changes by the systems, with a focus on selective data retrieval. It also centres on the health of the data as the primary focus and the developed tools for managing the complete ML life cycle. Emphasizing open questions, potential challenges, and upcoming research pathways, this review highlights the key importance of transitioning to seamless, automated data-centric systems in maintaining dependable and trustworthy ML systems in operational use, surpassing mere best practices.

Keywords: *Continuous data curation, Data valuation, Data drift, Model health, MLOps.*

Introduction

The integration of machine learning (ML) into nearly every domain, whether in science, industry, or everyday activities, has been motivated and encouraged by the abundance of large datasets and sophisticated computing systems. ML has been integrated into fields such as medical diagnostics and self-driving cars. In such domains, the deployment of ML systems does not have the luxury of waiting to be in a stable state. Reliability is not just something one can wish for; it is paramount (Naser, 2026). Yet the data fueling these systems is often of the quality one would not aspire for. Assuming that the data will remain as it was once collected is one of the greatest errors (the static-world assumption). The world is a dynamic and constantly changing entity, and it serves as the foundation for data and the processes that generate it (the data-generating processes). Predictably, this leads to a decline in all the system's attributes (e.g., quality, robustness, fairness) over time. The problem has been labelled as "AI ageing" (Vela et al.,

Article History: Received: 24.11.2025 Revised: 15.12.2025 Accepted: 18.12.2025 Published: 19.12.2025

Citation: Hasan, M., Shifa, S.I., Niaz, K., & Shuvo, M.M.H. (2025). Continuous Data Curation and Valuation for Long-Term Machine Learning Model Health: A Comprehensive Review. *European Journal of Science and Modern Technologies, 2*(1), 58-78. https://doi.org/10.59324/ejsmt.2026.2(1).05

2022). The processes of "ageing" of AI systems bring with them several sustainability and trust issues that many people assume have been mitigated, when in fact they have not.

The challenges at hand originate from the data used for the modelling. Achievement of data quality is not a one-off event. It is an iterative process. Noise, missing values, outliers, and incorrect labels cause great harm to the methods of training the model and the model's ability to generalise later on (Guha et al., 2024). More insidiously, the statistical properties of data can change over time, a phenomenon commonly referred to as "concept drift" (Lu et al., 2018). Such drift can take place in the form of changes in the distribution of the input data (covariate drift), the relationships between inputs and outputs (covariate drift), or the prior probabilities of the classes (label drift). This divergence between the production data and the training data leads to the model's internal representations becoming out of sync with reality, resulting in a loss of generalization. This is not hypothetical, and data from studies show that a significant percentage of ML models tend to experience performance decay when in production, and in some cases, within a few days of deployment (Nanny, 2023).

This challenge is the primary motivating factor for the need for ongoing data curation and valuation. Continuous curation is the continuing process of cleansing, enriching, and maintaining data quality throughout the ML lifecycle. It extends beyond the traditional, pre-deployment data preparation stage and incorporates data quality management as a continuous process within the operational MLOps pipeline. Complementing curation, data valuation is the process of determining the contribution of each data point to the model's performance. Knowing data value enables organisations to assess what data to acquire, annotate, and prioritise for curation, thus optimally channelling resources and improving model performance.

Nonetheless, the value of these particular data-centric practices is undoubtedly high, yet current ML systems and architectures still lack the necessary integration and automation. This results in data work being trivialised and viewed as a simple, manual, and janitorial task, rather than being treated with the dignity of being a first-class citizen of the ML lifecycle (Bhardwaj et al., 2024). This leads to the construction of brittle pipelines, in which data quality issues are often only discovered late in the process, if at all, and remediation is a costly and reactive endeavour. Although various data validation, drift detection, and active learning tools are available, integration of such tools into a unified, end-to-end data lifecycle management system remains largely unexplored and ineffective in real-world applications. The presence of these models continues to project an ML paradigm in which model-centric, algorithmic novelties are prioritised, as opposed to the foundational data that determines model performance.

This paper proposes DataSphere, a unified framework for autonomous data lifecycle management that consolidates automated curation, data valuation, drift detection, and active learning into a single data-centric pipeline for long-term model health. Model health is formally defined as a time-dependent function of accuracy, robustness, fairness, and distributional drift, moving beyond static evaluation metrics. A structured taxonomy is introduced to distinguish continuous curation tasks related to data integrity and data relevance, enabling systematic design of adaptive data pipelines. Finally, an implementation-oriented roadmap is provided to operationalize continuous curation and valuation in production machine learning systems, supporting sustained model reliability under evolving data conditions. By bringing together these disparate threads of research, this paper charts a course toward more robust, reliable, and sustainable ML systems.

This conceptual diagram illustrates the cyclical nature of the modern ML data lifecycle. It begins with data acquisition, followed by continuous curation (cleaning, deduplication, label correction). The assembled information is utilised for model training and validation. Once operational, drift and performance degradation tracking involves monitoring the model's predictions and the subsequent incoming data to identify any discrepancies. For retraining or selective acquisition, data valuation

techniques applied determine high-value data and maintain the cycle that simultaneously refines the data and the model for perpetual health.

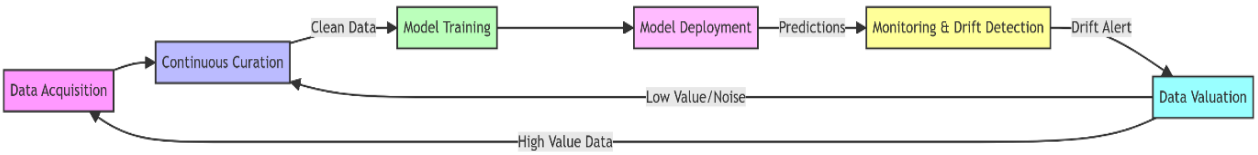

Figure 1. High-Level ML Data Lifecycle Pipeline

Foundations of Long-Term Model Health

The "health" of machine learning models is concerned with more than achieving a particular accuracy measure. It is a more comprehensive assessment of the model's performance, reliability, and robustness over time. A model is considered healthy if, in addition to executing its operational tasks accurately, it also remains stable and trustworthy within its surroundings as it changes. This section lays the groundwork for our discussion by defining model health, examining the mechanisms of its decay, and identifying the primary sources of its degradation. To provide a structured overview of the methodologies discussed in this review, This Study presents a master taxonomy of the techniques used to maintain long-term model health (Table 1).

Table 1. Master Taxonomy of Methods for Long-Term Model Health

Dimension	Category	Methods	Key Trade-offs
Data Quality	Curation	Rule-based Validation, ML-based Error Detection, Automated Label Correction	Precision vs. Automation: Rule-based is precise but brittle; ML-based scales but can introduce bias.
Data Importance	Valuation	Data Shapley, Influence Functions, Gradient-based Metrics	Accuracy vs. Cost: Shapley is theoretically rigorous but NP-hard; Gradients are fast but heuristic.
Environment Change	Drift Detection	DDM, ADWIN, Page-Hinkley, PSI, K-S Statistic	Sensitivity vs. Stability: High sensitivity catches fast drift but raises false alarms; low sensitivity misses gradual decay.
Adaptation	Acquisition	Uncertainty Sampling, Diversity Sampling, RL-based Active Learning	Labeling Cost vs. Performance: Aggressive sampling improves models faster but consumes the budget; conservative sampling saves costs, but risks lag.

Model health can be defined as the sustained ability of an ML model to deliver accurate, fair, and reliable predictions on production data over its entire operational lifecycle. This definition implies a multi-dimensional evaluation. Accuracy, while fundamental, is just one component among many. Robustness refers to the model's resilience to noisy, adversarial, or out-of-distribution inputs. Fairness ensures that the model's predictions do not disproportionately harm or benefit specific demographic groups. Reliability speaks to the consistency and predictability of the model's behaviour. A truly healthy model maintains a high standard across all these dimensions, not just at the time of deployment, but continuously throughout its entire production life.

The most common symptom of declining model health is accuracy decay, which is the gradual and sometimes sudden decline in predictive performance. This decay is a direct consequence of the model's static nature clashing with a dynamic world. As documented by Vela et al. (2022), this "AI ageing" is a complex phenomenon where the statistical relationships learned during training become obsolete. The rate of decay can vary significantly depending on the volatility of the data environment. In domains like finance or social media, where trends change rapidly, decay can be swift. In more stable environments, it may be a slow and insidious process. Decline that is harder to detect.

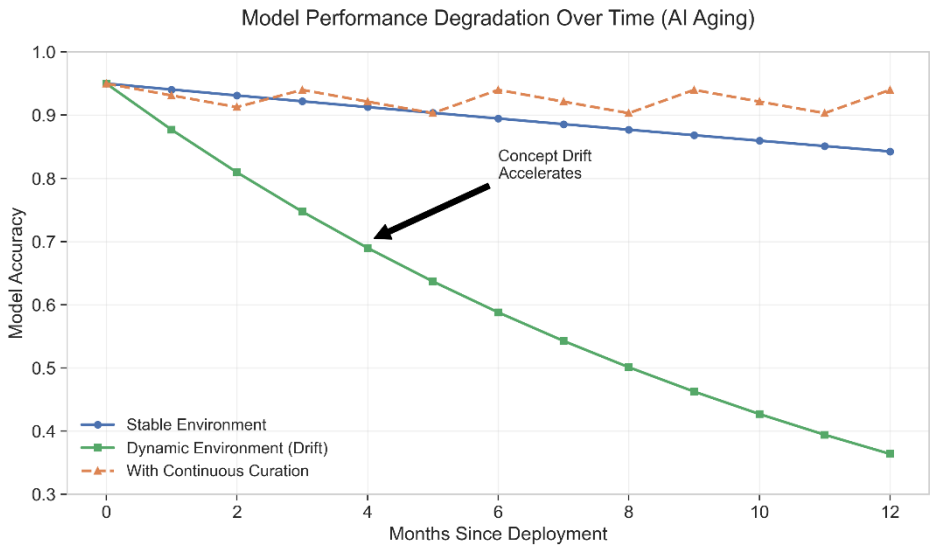

Figure 2. Model Performance Degradation Over Time (AI Aging)

This graph illustrates three scenarios of model health over time. The blue line represents a stable environment with slow natural decay. The red line represents a dynamic environment where concept drift accelerates performance loss, potentially leading to critical failure if left unchecked. The green dashed line illustrates the effect of continuous curation and periodic retraining, which maintains model performance near peak levels, resulting in a "sawtooth" pattern of stability. Based on concepts from Vela et al. (2022).

Beyond simple accuracy, robustness issues are a critical aspect of model health degradation. A model that is accurate on clean, well-formed data may fail catastrophically when faced with the messiness of real-world inputs. This includes not only data drift but also data quality issues, such as missing values, sensor noise, or formatting errors, that were not adequately represented in the training set. The sources of this degradation are manifold. The most prominent is data drift, where the statistical properties of the input data change over time. This can be covariate drift (changes in the distribution of independent variables) or concept drift (changes in the relationship between independent and dependent variables) (Lu et al., 2018). Another source is data quality erosion, where upstream data pipelines introduce errors, or data collection practices change. Finally, selection bias can occur, where the data used for training is not representative of the population the model will encounter in production, leading to systemic performance gaps.

Formalising Long-Term Model Health

To quantify model health rigorously, it is useful to view it not as a single scalar score, but as a time-dependent function governed by multiple interacting factors. Let $H(t)$Denote the health of a model at

time. t. Define model health as a function of several state variables that jointly characterise performance and reliability:

$$H(t) = f\big(A(t), R(t), F(t), D(t)\big) \tag{1}$$

Here, $A(t)$represents predictive accuracy, measured using task-appropriate metrics such as the F1-score or AUC on the current production data distribution $P_t(X, Y)$. Robustness, denoted by $R(t)$, captures the model's stability under small input perturbations $\epsilon$and can be expressed as

$$R(t) = \mathbb{E}_{(x,y)\sim P_t}\big[\mathbb{I}\big(M(x) = M(x+\epsilon)\big)\big] \tag{2}$$

Fairness, $F(t)$, reflects disparities in model outcomes across protected groups and may be quantified using standard group fairness metrics such as the Equalised Odds difference:

$$F(t) = \left| P\big(\hat{Y} = 1 \mid Z = 0\big) - P\big(\hat{Y} = 1 \mid Z = 1\big) \right| \tag{3}$$

Finally, $D(t)$measures distributional drift by quantifying the divergence between the training data distribution $P_{\text{train}}$and the current data distribution P_t, commonly using the Kullback–Leibler divergence:

$$D(t) = D_{KL}(P_{\text{train}} \parallel P_t) \tag{4}$$

Within this formulation, *AI ageing* can be naturally interpreted as a decline in model health over time, corresponding to the condition. $\frac{dH(t)}{dt} < 0$. In practice, this degradation is often driven by increasing drift. $D(t)$, which in turn negatively affects both accuracy $A(t)$and robustness $R(t)$.

In parallel, the objective of data valuation can be formalised as assigning a scalar value. $V_i$to each training instance $(x_i, y_i) \in S$, reflecting its marginal contribution to the overall utility U Of the model. A principled approach to this problem is provided by the Shapley value, defined as

$$V_i = \frac{1}{|S|}\sum_{S' \subseteq S\setminus\{i\}} \binom{|S|-1}{|S'|}^{-1} [U(S' \cup \{i\}) - U(S')] \tag{5}$$

where $U(S)$ is the utility corresponding to model M trained on a subset? S The data is usually obtained through validation. There is a lot of theory that is strongly backed as valid for this specification, but this formulation has one major practical difficulty: calculating V_i Precisely, one has to evaluate the model on an exponentially large number of data subsets. This is a concerning estimation intractability, which is the reason this paper studies techniques to approximate Vi. These techniques are the focus of Section 4.

Continuous Data Curation

As the first and foremost layer to bring model curation to a sustainable model, Continuous Data Curation signals a shift from traditional one-off data preparation to an integrated, continuous process performed throughout the entire ML lifecycle. It is the constant and automated supervision of data quality, ensuring that the data used to fuel ML models is clean, consistent, and relevant. This section defines continuous data curation, describes its primary functions, and summarizes the automated systems that enable its large-scale application.

Continuous data curation can be defined as the set of automated processes and practices for systematically identifying and rectifying quality issues in a data stream that is used for training and monitoring machine learning models. Unlike static data cleaning performed before initial model training, continuous curation operates within the live MLOps pipeline. It treats data not as a fixed asset but as a dynamic entity that requires constant vigilance. The goal is to create a self-healing data pipeline that proactively detects and mitigates issues, such as schema changes, data entry errors, outliers, and label noise, before they can negatively impact model performance. This practice is a direct response to the

understanding that data work is not a preliminary step but a core, recurring component of successful ML systems (Bhardwaj et al., 2024).

The practice of continuous data curation encompasses several critical tasks. Data cleaning is the most fundamental of these, involving the detection and correction of inaccuracies and inconsistencies. Modern automated cleaning tools leverage a variety of techniques, from simple rule-based validation to sophisticated ML-based error detection (Mumuni & Mumuni, 2024). For instance, deep learning models can be trained to identify subtle anomalies in large datasets that would be impossible for humans to find manually. Data deduplication is another crucial task, particularly in large-scale systems, where redundant data can skew model training and lead to inefficient resource utilisation. Advanced deduplication goes beyond finding exact duplicates, using ML to identify semantic duplicates—records that refer to the same entity but are represented differently. Label correction is a specialised but vital form of curation, particularly in supervised learning. Mislabeled training data can significantly degrade model performance. Automated techniques for label correction, such as those found in frameworks like Cleanlab, use ML models to identify labels that are likely to be incorrect by analysing model confidence scores and other signals, flagging them for review or even correcting them automatically (Northcutt et al., 2021).

This bar chart in Fig 3 compares the robustness of standard training with training that incorporates automated label correction. As the percentage of mislabeled data increases (x-axis), the accuracy of a standard model (green bars) drops precipitously. In contrast, a model trained with an automated label correction pipeline (blue bars) maintains significantly higher accuracy, demonstrating the resilience provided by continuous curation. Based on findings from Northcutt et al. (2021).

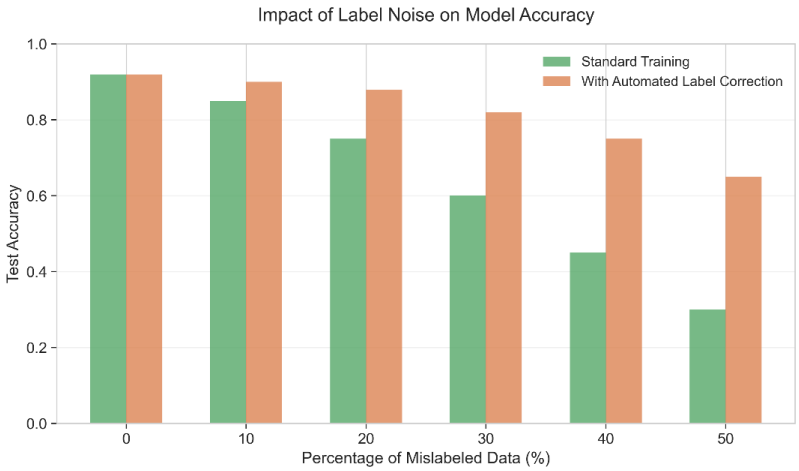

Figure 3. Impact of Label Noise on Model Accuracy

Automated curation frameworks are the engines that power this continuous process. These frameworks are designed to be integrated directly into data pipelines, often running on distributed computing platforms such as Apache Spark, to handle large volumes of data. They provide a declarative language for defining data quality constraints, also known as "expectations." For example, a user might declare that a specific column should never contain null values, that its values must fall within a certain range, or that its distribution should match a reference profile. The framework then automatically validates incoming data against these expectations, generating detailed data quality reports and quarantining or repairing data that fails validation. These frameworks often include components for data profiling, which automatically generate a statistical summary of the data, helping to bootstrap the process of creating

expectations. By codifying data quality as a form of testing, these frameworks bring a DevOps-like rigour to data management, enabling a "data quality as code" approach.

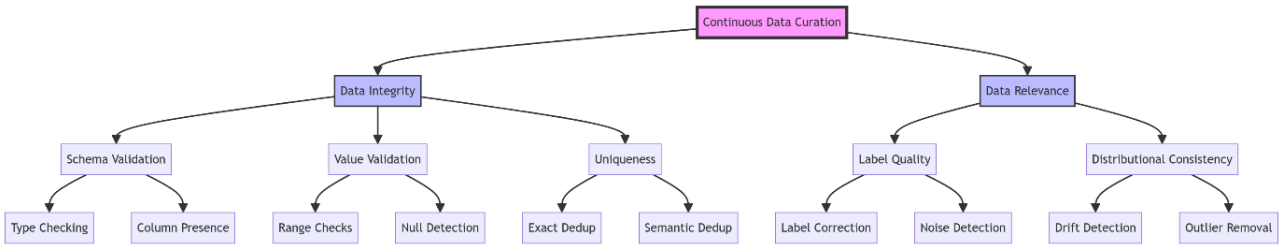

Figure 4. Taxonomy of Curation Tasks

This diagram organises the core tasks of continuous data curation into a hierarchical structure. At the highest level, curation is divided into two main branches: Data Integrity and Data Relevance. Under Data Integrity, sub-branches include Schema Validation (checking data types and column presence), Value Validation (range checks, pattern matching, and null detection), and Uniqueness (deduplication of data). Under Data Relevance, sub-branches include Label Quality (label correction and noise detection) and Distributional Consistency (drift detection and outlier removal). This taxonomy offers a structured framework for evaluating the various aspects of data quality that must be continually monitored and managed.

Table 2. Comparison of Data Curation Techniques

Technique	Description	Key Strengths	Key Weaknesses	Representative Tools/Papers
Rule-Based Validation	Validates data against predefined constraints (e.g., ranges, types, nulls).	Simple to implement, interpretable, deterministic.	Brittle, requiring manual rule definition, and prone to missing complex errors.	Great Expectations (Schelter et al., 2018), Deequ (Schelter et al., 2018)
Outlier Detection	Identifies data points that deviate significantly from the statistical distribution.	Can detect unknown error types, useful for drift detection.	High false positive rate, sensitive to threshold choice.	PyOD, Isolation Forest
ML-Based Error Detection	Uses trained models to predict errors or inconsistencies in data.	Can capture complex dependencies, adaptable to new data patterns.	Requires training data, opaque (black-box), and computationally expensive.	HoloClean, Raha
Automated Label Correction	Uses model confidence or ensemble agreement to identify and fix mislabels.	Directly improves supervised learning performance and reduces labelling cost.	It can introduce bias if the correction model is flawed.	Cleanlab (Northcutt et al., 2021), Confident Learning
Entity Resolution (Deduplication)	Identifies and merges records referring to the same real-world entity.	Essential for data integrity in multi-source systems.	Computationally intensive (cap O open paren capsive ($O(N^2)$), difficult to tune for precision/recall.	Zingg, Deep Matcher
ML-Based Label Correction	Uses model predictions and confidence scores to identify and fix mislabeled data.	Improves label quality and supervised model performance.	Complex to implement; may introduce errors if poorly tuned.	Cleanlab; Nguyen et al. (2023)

Semantic Deduplication	Uses embeddings or clustering to detect records referring to the same entity.	Finds non-obvious duplicates; robust to format variation.	Requires labelled data or advanced unsupervised methods.	Zingg; Dedupe.io

Data Valuation in Machine Learning

While continuous curation ensures data quality, data valuation addresses a complementary and equally crucial question: what is the contribution of each data point to a model's performance? Not all data is created equal; some points are redundant, some are noisy, and some are exceptionally informative. Data valuation provides a principled framework for quantifying the worth of individual data points, enabling a more strategic approach to data acquisition, labelling, and management. This section delves into the meaning of data value, reviews the primary methodologies for its calculation, and discusses its role in industrial ML systems.

Data points have no intrinsic worth. Their value is relative and highly contextual to the learning algorithm being used, the performance metrics of interest, and the other samples used during training set (Ghorbani & Zou, 2019). A point that may mean everything to the training of a Logistic Regression model may mean nothing to a deep neural network. Likewise, this value is influenced by whether the aim is to maximise accuracy, fairness, or robustness. The essence of data valuation is to go beyond the heuristics and construct formal, justifiable frameworks that ascribe a model's success or failure to the data used to train the model. This is what leads a data lifecycle to become optimal and more cost-efficient by concentrating efforts on the data that is most valuable.

One of the most fundamental contributions to data valuation is the data Shapley framework by Ghorbani and Zou (2019). This model employs the Shapley value from cooperative game theory, which proposes a fair and unique approach to dividing the total "payout" (i.e., the model's performance) among the "players" (i.e., data points). The shapely value of a data point is defined as its average marginal contribution to the attained performance across all the possible subsets of the training data. It has been established that this approach possesses strong theoretical benefits, as it is the only method of valuation to meet the criteria for a variety of desirable features, such as symmetry (where two points with equal contributions have equal assessed worth) and additivity. From a practical perspective, Data Shapley has also proven to be a highly effective method. Ghorbani and Zou have shown that it outperforms other, less complicated techniques such as leave-one-out (LOO) scoring, which was designed specifically for determining the worth of data. Moreover, data points with low or negative Shapley values often represent outliers, erroneous labels, or corrupted data, which is why they are a strong indicator of a collection. The most prominent disadvantage of the Shapley value is its computational aspect, as calculating it precisely would require an infeasibly large amount of time. However, considerable efforts have been made to resolve the computational burden associated with accurately approximating the Shapley value. Near the top of this list are methods based on Monte Carlo sampling and gradient techniques, which have successfully opened the door to a variety of applications.

The histogram depicts a typical data distribution. Most data points (centre mass) contribute a small value to the model's accuracy. However, a small subset of data points on the right (the tail) are high-value data critical to performance. The distribution also indicates a left tail of negative Shapley value data points. These data points are detrimental to the model (e.g., outlier data points or mislabeled examples, and their removal would enhance the model. Concepts from Ghorbani and Zou (2019) are used here (Nguyen et al., 2023).

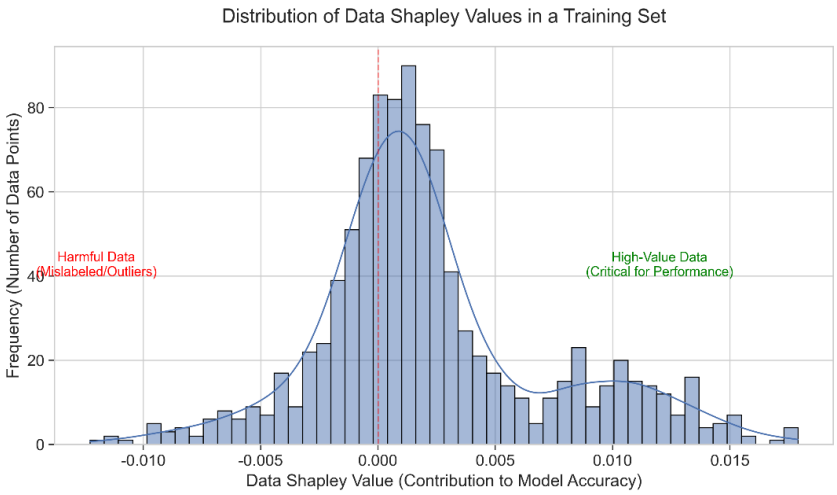

Figure 5. Distribution of Data Shapley Values in a Training Set

Another technique for valuing data is influence functions, a method in robust statistics that Koh and Liang (2017) adapt for modern machine learning in "Influence Functions for Nonparametric Learning". An influence function considers the impact of upweighting a single training observation on the model parameters and, therefore, the predictions. In essence, it considers what model parameter would look like if that particular data point were not in the training set and the model were retrained, minus the retraining cost. This is done by estimating the influence of the training point on the model loss for a given test point. Influence functions have provided a way to retrieve predictions from black box models, thereby tracing specific predictions to the training points most responsible for them. This is useful for model debugging, explaining model behaviour, and identifying data artefacts or biases. If a model generates a peculiar prediction, influence functions indicate to a user the training examples that were influential in the model's prediction, which may be out-of-sample observations or training examples that were misclassified.

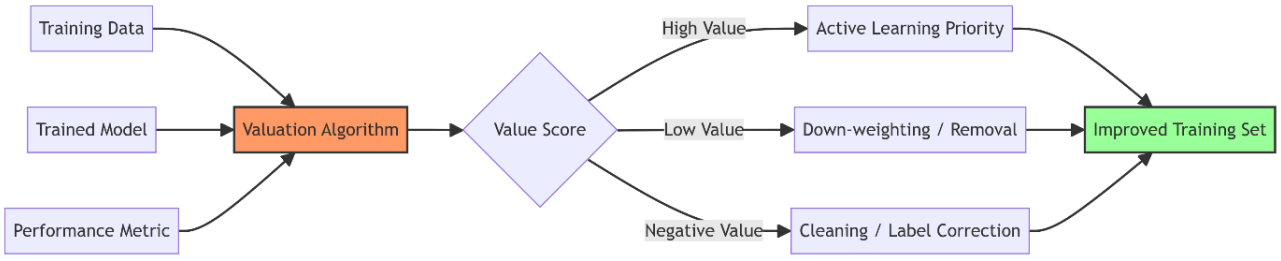

Figure 6. Conceptual Pipeline of Data Valuation

In addition to these foundational approaches, other methods have been developed. At the same time, less theoretically elegant, gradient-based metrics take a more computationally sleek approach. Data Valuation with Gradient Similarity (DVGS) operates on the assumption that training examples with gradient vectors clustered in the same direction are more informative, positing that data points are more valuable if they move the model in a similar direction (Evans & Liu, 2024). Signals from active learning can also be viewed as a form of implicit, forward-looking data valuation. In active learning, a model queries the labels of the most informative (according to its internal criteria) unlabeled data points. In

strategies like uncertainty sampling, where the model requests labels for the points on which it is most uncertain, data that can effectively resolve the model's ambiguity to alter its decision boundary is assigned a high value.

This diagram shows data valuation steps in an integrated MLOps pipeline. The pipeline begins with a trained model along with some performance measures/metrics. Valuation algorithms (e.g., data Shapley, influence functions) take the training data with the model and output a value score for individual data points. The value of the score is then used to make a decision. For example, flagged low-value data may be removed or down-weighted, while high-value data may be used to inform active learning or data acquisition strategies. Data points with negative value scores may be prioritised for cleaning and label correction. This enables a feedback loop and an iterative process for improving the training set.

Table 3. Summary of Major Data Valuation Techniques

Technique	Principle	Key Strengths	Key Weaknesses	Primary Use Case
Data Shapley	Cooperative game theory (marginal contribution).	Theoretically rigorous, it satisfies fairness axioms (symmetry, additivity).	Computationally expensive (NP-hard), requires approximation.	Data cleaning, equitable data pricing.
Influence Functions	Robust statistics (Hessian-based approximation).	Efficiently approximates the retraining impact, making it excellent for debugging.	Assumes convex loss, unstable for deep learning (non-convex).	Debugging model predictions, identifying outliers.
Gradient Similarity	Similarity of gradients between training and validation data.	Computationally fast and scales well for deep learning.	Heuristic-based, lacks the theoretical guarantees of Shapley.	Coreset selection, fast data pruning.
Reinforcement Learning	Learn a data selection policy to maximise reward (accuracy).	Can optimise for arbitrary non-differentiable metrics.	Hard to train, unstable, high variance.	Automated curriculum learning.
Beta Shapley	Weighted Shapley value emphasising low/high cardinality subsets.	More flexible than standard Shapley, it can prioritise specific data types.	Inherits computational complexity of Shapley.	Specialised valuation tasks.
Active Learning	Implicitly values unlabelled data by estimating its potential impact if labelled.	Directly optimises model improvement and reduces labelling cost.	Limited to unlabelled data; value depends on the query strategy.	Cost-effective data labelling and acquisition.

Drift Detection & Adaptive Feedback Loops

The recognition that data distributions are not static is fundamental to long-term model health. Drift detection serves as the nervous system of a data-centric ML pipeline, providing the critical signals that the world has changed and the model may no longer be reliable. When integrated with the curation and valuation mechanisms discussed previously, drift detection enables the creation of adaptive feedback loops that allow systems to autonomously maintain their performance. This section explores the primary types of data drift, reviews the major algorithms for its detection, and discusses their integration into retraining cycles.

Data drift, in its broadest sense, refers to a change in the underlying data distribution between the training environment and the production environment. This change can be categorised into several types. Concept drift is perhaps the most challenging, involving a change in the relationship between the input variables and the target variable (i.e., $P(y|X)$). For example, in a fraud detection system, the patterns of fraudulent behaviour may evolve, rendering the original model obsolete. Covariate drift, also known as

data drift, refers to a change in the distribution of the input variables themselves (i.e., P(X)). A model trained on customer data from one geographical region may perform poorly when deployed to another with different demographics. Other types include label drift, a change in the prior distribution of the classes (P(y)), and feature drift, where the meaning or relevance of individual features changes over time. These drifts can be sudden and abrupt, caused by events such as sensor malfunctions or changes in the user interface, or they can be gradual and incremental, reflecting evolving user behaviours or seasonal trends (Lu et al., 2018).

The timely detection of these drifts is paramount. A variety of algorithms have been developed for this purpose, many of which originate from the field of statistical process control. The Drift Detection Method (DDM) is a classic online approach that monitors the model's error rate (Gama et al., 2004). It maintains a window of recent predictions and raises a drift alarm if the error rate exceeds a certain threshold, defined by the expected variance of the binomial distribution of errors. The Early Drift Detection Method (EDDM) is a refinement of DDM that is more sensitive to gradual drifts by monitoring the distance between two consecutive errors (Baena-García et al., 2006). For unsupervised drift detection on input features, the Page-Hinkley Test is a sequential analysis technique that monitors the cumulative difference between a variable's running average and its global average, signalling a drift when this cumulative sum exceeds a predefined threshold (Page, 1954). More advanced methods, such as ADWIN (Adaptive Windowing), dynamically adjust the size of the window being monitored, allowing them to adapt to different rates of change and providing mathematical guarantees on their false positive and false negative rates (Bifet & Gavalda, 2007). These algorithms form the first line of defence, providing the quantitative signals that trigger further investigation and action.

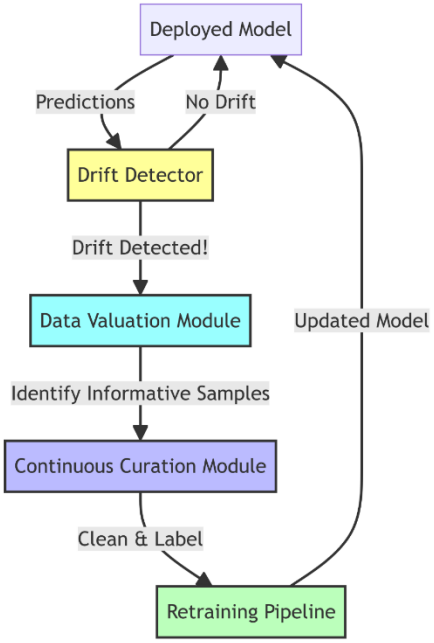

Figure 7. Drift–Curation–Valuation Feedback Loop

The integration of drift detection into the broader ML lifecycle is what creates a truly adaptive system. When a drift detector raises an alarm, it should not be an isolated event but the start of a coordinated response. The first step is typically to trigger a deeper analysis to understand the nature and scope of the drift. This may involve using data valuation techniques to identify which data points are contributing

most to the drift. For example, influence functions can be used to trace the increased error rate back to specific subsets of the new data that the current model poorly handles. Once the drift is understood, the system can initiate a retraining cycle. However, simply retraining on all the new data may not be the optimal solution. This is where the feedback loop with curation and valuation becomes critical. Data valuation can be used to selectively sample the most informative new data for retraining, optimising the use of labelling and computing resources. Continuous curation ensures that this new data is cleaned and validated before it is used for training, preventing the model from learning from low-quality or corrupted inputs. This creates a closed-loop system where drift is not just detected but is used as a signal to intelligently and automatically improve both the data and the model.

This diagram visualises the integrated, cyclical process of maintaining model health. The loop begins with a deployed model making predictions on live data. A Drift Detector continuously monitors the input data and the model's output (e.g., accuracy). When a significant drift is detected, an alert is triggered. This alert activates the Data Valuation module, which analyses the new, drifted data to identify the most informative or problematic samples. The output of the valuation module then feeds into the Continuous Curation module, which cleans, validates, and prepares the selected data for further processing. This curated dataset is then used to retrain or update the model. The updated model is deployed, and the cycle begins anew.

This feedback loop transforms the ML system from a static artefact into a dynamic, self-adapting organism.

Selective Data Acquisition & Active Learning

In a data-centric ML paradigm, the process of acquiring new data is as strategic as the process of training the model itself. Rather than passively accepting all available data, selective data acquisition aims to identify and obtain the most informative data to improve a model's performance, fairness, or robustness most cost-effectively. Active learning is the primary mechanism through which this is achieved, creating an intelligent dialogue between the model and the data source. This section discusses the importance of selective data collection and reviews the principal strategies used in active learning.

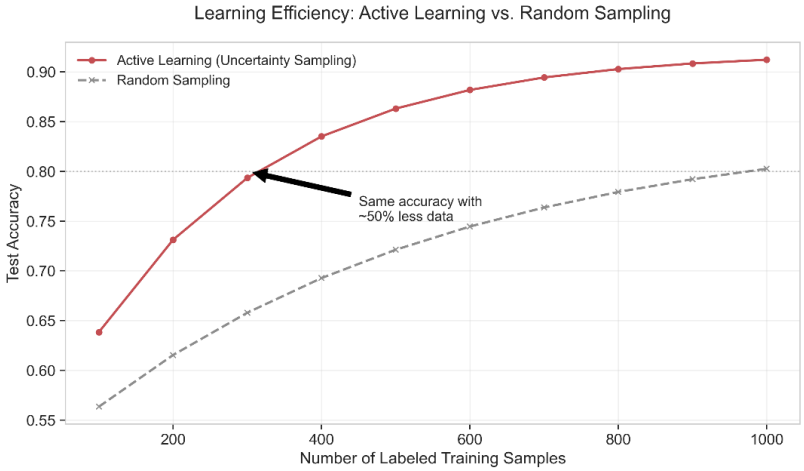

Figure 8. Drift–Curation–Valuation Feedback Loop

The economic and efficiency considerations give rise to the concept of selective data acquisition. The ML lifecycle entails multiple processes, and unfortunately, data labelling is the most resource- and time-demanding of them all. In many scenarios, the majority of the data available is not labelled and,

unfortunately, it is redundant. The model trained on it does not gain or learn anything new. In contrast, the data that the model is most "curious" about is the only data that should be considered. The labelling costs are dramatically reduced, and the model trained on it performs equally, or even better, than a model trained on a larger, randomly sampled dataset (Attenberg & Provost, 2011). This is especially important when models require frequent updates. High-value data enables the organisation to accelerate the adaptation cycle and maintain the model's efficiency.

The learning curve illustrates the performance of models trained using Active Learning (Uncertainty Sampling) compared to models trained with Random Sampling. With less labeled data, Active Learning (blue line) achieves notably more accuracy than Random Sampling (grey dashed line). The annotation states that Active Learning is capable of reaching the performance target (e.g., 80% accuracy) with 50% less data, which exemplifies the increased cost efficiency of selective acquisition and is supported by well-known benchmarks (e.g., Settles, 2009; Bi et al., 2025).

Active learning frameworks formalise this process. In a typical active learning cycle, a model is trained on a small initial set of labelled data. It then examines a large pool of unlabeled data. It employs a query strategy to select the most informative sample to be labeled by a human oracle (or an automated labeling service). The cycle is repeated once the model is updated, and this freshly labeled sample is added to the training set. The sophistication of the query method is the primary factor that determines the effectiveness of active learning.

Uncertainty sampling is the most common and intuitive family of query strategies. The model queries the data points for which it is least certain about its prediction. There are several methods for measuring this uncertainty. For probabilistic models, least confident sampling selects the instance with the lowest predicted probability among all cases. Margin sampling selects the instance where the difference between the probabilities of the two most likely classes is smallest, indicating that the model is torn between two choices. Entropy-based sampling selects the instance with the highest entropy across its predicted probability distribution, indicating maximum confusion. These methods are computationally efficient and have proven effective in a wide range of applications (Raj & Nagi, 2022).

Although powerful, uncertainty sampling can sometimes be myopic, selecting an entire batch of similar, uncertain points. To remedy this, diversity sampling selects a batch that is uncertain but also diverse, representing different areas of the feature space. This is done by combining an uncertainty metric with a diversity metric. For instance, the unlabeled data can be clustered, and different clusters can contribute a sampled, uncertain point. In this manner, the model learns about other parts of the problem space and does not overfit a specific region of the decision boundary.

More advanced strategies employ techniques from reinforcement learning (RL) and multi-armed bandits. In this framework, the active learning agent learns a policy for selecting which data points to choose. The "actions" are the data points to query, and the "reward" is the resulting improvement in model performance. By exploring different query strategies and exploiting those that yield the highest rewards, these RL-based approaches can learn a data acquisition policy optimized for a specific model and dataset, often outperforming fixed, heuristic-based strategies (Chai et al., 2022).

Table 4. Comparison of Active Learning Strategies

Strategy	Principle	Key Strengths	Key Weaknesses	Typical Application
Uncertainty Sampling	Select points the model is most uncertain about (e.g., low confidence,	Simple, computationally efficient, and effective	It can be myopic, may select outliers, and is less effective for regression.	Text classification, image classification.

	small margin, high entropy).	for many classification tasks.		
Query-by-Committee (QBC)	Train a committee of models and select points where the models disagree the most.	Robust, less sensitive to the biases of a single model.	Computationally expensive due to the need to train multiple models.	When model robustness is critical.
Diversity Sampling	Select a batch of points that are both uncertain and diverse in feature space.	Avoids redundant queries and provides a more comprehensive view of the data distribution.	It can be more complex to implement and requires a meaningful distance metric.	Large-scale labelling tasks where batch selection is needed.
Reinforcement Learning	Learn a policy to select data points that maximise the long-term reward (model performance).	Can adapt its strategy to the specific dataset and model, potentially optimal.	Very complex, requiring significant data to learn an effective policy, and can be unstable.	Highly specialised domains where the labelling budget is extremely limited.

Data Health Monitoring

In relation to data-related illnesses, if there are two treatments, continuous curation and drift detection, then health checking on data is the system for initial diagnosis. It is the practice of continuously computing and monitoring some metrics on the data in relation to its ongoing quality, integrity, and stability. This section explains the concept of data health, reviews key metrics of quality, and examines data monitoring solutions that have begun to automate this crucial function.

Data health is a measure of the overall condition of data within a dataset, including its composition, structure, and whether it is in a statistically consistent state, as well as whether it remains so for a specific duration of time. A healthy data set has no structural issues, fits within the data set/framework of the statistical framework's expected relationships, is stable, and does not change unpredictably over time. Expanding from a one-off validation exercise to a more automated and continuous system that provides a real-time dashboard of a dataset's vital signs enables more advanced monitoring of data health. This will allow data teams and ML engineers to flag concerns at their source, and do so much earlier in the model's lifecycle, in a timeframe that should precede a drop-off in the model's performance line. In the broad sense, there are two types of metrics when working on monitoring data health. Structural metrics refer to the scheme and format of the data, including checks on data types, the presence of required columns, and counts of null values. Statistical metrics refer to the population of values in the data and measure the distribution of such values. This includes basic summary statistics (mean, median, standard deviation) as well as more advanced measures, such as quantiles, histograms, and correlations between features. Metrics of consistency capture how these structural and statistical attributes of a dataset evolve. Here is where monitoring data health overlaps with drift detection using the Kolmogorov-Smirnov (K-S) statistic and the Population Stability Index (PSI) to measure the distance between the current data distribution and a baseline distribution (e.g., training set data).

Open-source software products to monitor and manage the overall health of enterprise data have gained considerable popularity over the last few years. Great Expectations (GX) ranks among the industry leaders, allowing users to define data quality checks in natural language, which the system refers to as 'Expectations' (Schelter et al., 2018). For instance, an end-user of the system can specify an expectation that all values in a given column are unique, or that the column's mean falls within a certain range. Thereafter, GX verifies the data against defined expectations. This system produces and disseminates

data documentation ("Data Docs") and can be connected to data pipelines to abort stages of a pipeline if faults in the data are identified. TensorFlow Data Validation (TFDV) is another provider in the same space. However, with a wider industry focus on integration and overall system architecture (scalability) with TensorFlow as part of the TFX ecosystem (Breck et al., 2019). TFDV is also capable of recommending an operational schema based on the training data, and the system will identify data types as well as the features that are present or absent in the specified ranges. An instance of the analytical tool can employ a given schema to identify anomalies in the served data by obtaining schema skew (a schema mismatch) and distribution skew (a change in data distribution). Another library based on the Apache Spark framework is Deequ, which supports the evaluation of multiple data quality metrics in massive datasets (Schelter et al., 2018). It empowers users through a highly extensible system to impose and satisfy a multitude of criteria, thereby achieving data quality in data-rich environments.

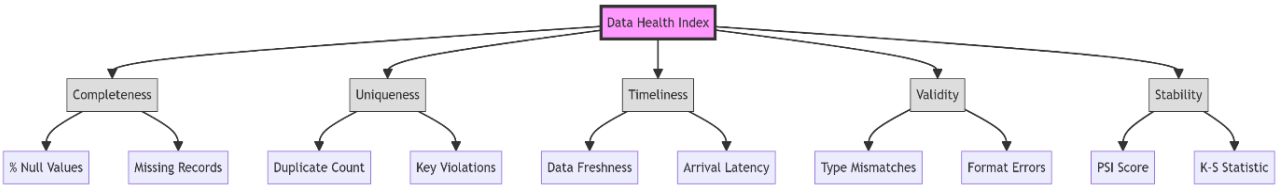

Figure 9. Data Health Index Structure

This exemplifies how numerous metrics of data quality can be consolidated to form a singular, albeit multi-faceted, conceptual Data Health Index. This index is a weighted average of the constituent indexes, which include, for example, Completeness (percentage of null values), Uniqueness (number of duplicates), Timeliness (adequate staleness of data), Validity (percentage of values which abide to the rules of type and format), and Stability (as determined by some drift metric, like the PSI/K-S statistic). Each of the indices mentioned is a function of several metrics lower than it. This highly stratified system enables a holistic view of data health and, if necessary, allows for the polarization of specific metrics, allowing users to identify and isolate problem areas.

Table 5. Data Health Tools Comparison

Tool	Key Features	Primary Ecosystem	Strengths	Weaknesses
Great Expectations	Declarative data testing, automated docs, pluggable backends.	Python, Airflow, dbt	Human-readable documentation, highly extensible, and a strong community.	It can be verbose to configure, and there is a performance overhead for large data.
TensorFlow Data Validation (TFDV)	Schema inference, drift detection, and skew detection.	TensorFlow Extended (TFX)	Scalable (Apache Beam), tight integration with the TF ecosystem.	Steep learning curve, heavy dependency on TF/Beam.
AWS Deequ	Unit tests for data, scalable metrics calculation.	Apache Spark, Scala/Java	Built for massive scale (Spark) with a declarative API.	Requires a Spark cluster, which is less user-friendly for Python users.
Evidently AI	Drift detection, model performance monitoring, and interactive reports.	Python, Pandas	Excellent visualisations, easy to use, and comprehensive drift metrics.	Primarily for monitoring (post-hoc), with less focus on pipeline testing.

Pandera	Runtime data validation for pandas/polars dataframes.	Python, Pandas	Lightweight, integrates with type hints, and zero-config schema inference.	Limited to single-node memory (mostly), with less enterprise tooling.

Toward Unified Frameworks

Each of the data-centric elements of an ML system – curation, valuation, drift detection, and monitoring – is highly valuable. Still, they are most beneficial when all are combined in a singular framework. The industry is shifting from separately functioning, disorganised tool sets to fully developed MLOps platforms that optimise all functions of the ML lifecycle. This section of the document focuses on the current state of integration, examines the architecture of fully developed systems, and compares it with that of leading industry firms.

Integration is greatly needed due to the drift in interconnectedness of the data lifecycle. A drift signal is only valuable when paired with a valuation and retraining pipeline. When data valuation can actively inform the components of learning and curation prioritisation, it is most efficient. A singular framework provides the orchestration layer needed to manage these dependencies, creating an unimpeded flow of data and control signals between components. This reduces manual hand-offs and, in turn, significantly accelerates the entire system, from data ingestion to model deployment and monitoring. It provides a comprehensive perspective of the model and the data's health; when a deviation is detected in one portion of the system, the rest can automatically resolve it.

The architecture of a unified framework, which might be termed a "DataSphere," is designed around the principle of data as a first-class citizen. Its centralised feature store, which maintains and versions features for both training and serving to guarantee consistency between the two contexts, is its essential component, surrounding numerous data-centric services. A Data Ingestion and Curation Service continuously pulls in raw data, validates it against predefined expectations, cleans it, and registers it in the feature store.

There exists a service for model training and valuation that coordinates the training of models and, most importantly, performs the algorithms for data valuation, rating the training data accordingly. Thus, for every training example, the value of the training data is computed and stored. Then this value is stored as metadata of that training record as a feature. A model deployment and Monitoring service is responsible for deploying the models as endpoints and performing continual drift detection and data health monitoring on the active traffic. When the monitoring service identifies a problem, it communicates the issue back to the training service, which can then utilise the stored value and incoming data to activate an intelligent retraining and curation process. This entire process is autonomous and controlled by a central orchestration module, typically developed using systems such as Kubeflow Pipelines or Apache Airflow. Major cloud vendors and MLOps companies are developing services that align with this consolidated vision. Google Cloud Vertex AI offers a managed service that combines a feature store, model monitoring, and pipeline orchestration, among other features (Baylor et al., 2017). They allow customers to build automated, end-to-end MLOps pipelines that can detect data drift and skew in addition to training, deploying, and monitoring models. For data preparation (SageMaker Data Wrangler), feature management and storage (SageMaker Feature Store), model monitoring (SageMaker Model Monitor), and workflow automation (SageMaker Pipelines), SageMaker offers an equally comprehensive set of tools (Liberty et al., 2020). These tools focus more on data management and data-related functionalities, as having the right data management is the foundational piece needed for production ML. Experiment tracking, model registries, and pipeline orchestration are functionalities that open-source tools like MLflow and Kubeflow also provide, allowing users to create custom, cohesive

frameworks that can be combined with other libraries for data quality and drift detection, and that offer the other functionalities needed to manage data in ML.

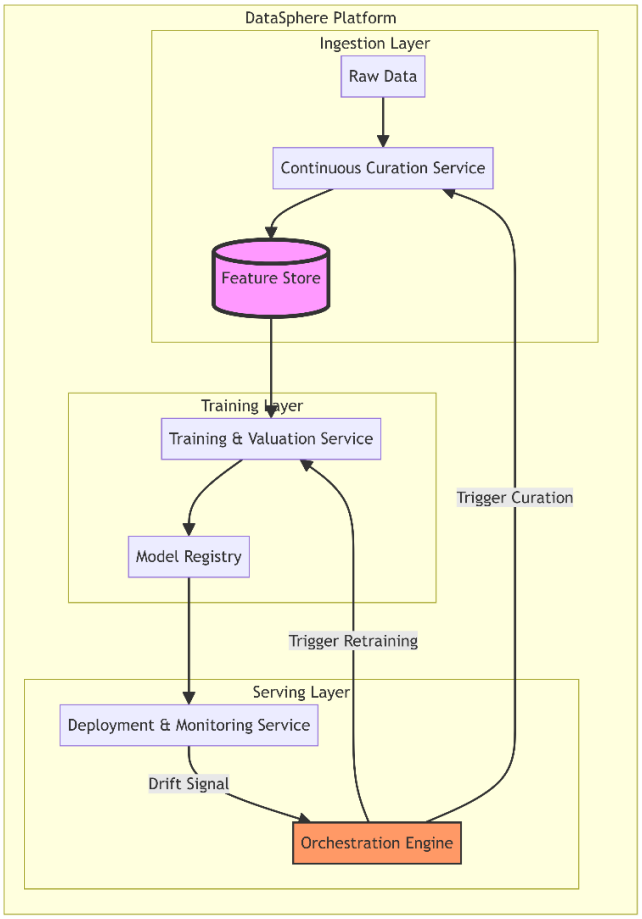

Figure 10. Unified DataSphere Architecture

An integrated, data-focused machine learning platform's high-level architectural plan is shown in this diagram. The Feature Store, the one source of truth for all features, is located at the centre. The system uses a Continuous Curation Pipeline to receive data. A training and validation pipeline consumes features from the store to train models and calculate data values. A deployment and monitoring pipeline serves models, tracking their performance and data drift in real-time. A central Orchestration Engine coordinates the flow between these pipelines, creating an automated feedback loop where drift detection triggers intelligent retraining and curation.

Table 6. Industrial ML Lifecycle Frameworks Comparison

Framework	Key Data-Centric Features	Integration Level	Strengths	Weaknesses
Google Vertex AI	Feature Store, Model Monitoring (drift/skew), Data Labelling Service.	High (Fully Managed)	Seamless integration with Google data stack (BigQuery), strong monitoring capabilities.	Vendor lock-in can be expensive for small-scale businesses.

Amazon SageMaker	Data Wrangler (prep), Feature Store, Model Monitor, Clarify (bias).	High (Fully Managed)	Comprehensive toolset, strong security/compliance, massive scale.	Complex UI/UX, steep learning curve, vendor lock-in.
Databricks (MLflow)	Delta Lake (versioning), Feature Store, MLflow (tracking).	Medium (Platform + Open Source)	Unified data/AI platform, open standards (MLflow), excellent for Spark users.	Monitoring/drift detection is less mature than cloud-native options.
Kubeflow	Pipelines for orchestration, Katib for tuning, and metadata tracking.	Low (DIY / Open Source)	Cloud-agnostic, highly customizable, runs on Kubernetes.	High operational complexity requires significant engineering to maintain.
Tecton	Enterprise Feature Store, real-time serving, historical backfill.	Specialised (Feature Store)	Best-in-class feature management supports batch/stream/real-time.	Focuses solely on the feature layer and requires integration with other MLOps tools.
MLflow + Kubeflow	Open-source stack integrated with tools such as Great Expectations and TFDV.	Medium (manual integration required)	Flexible, customizable, avoids vendor lock-in, and has strong community support.	High engineering effort; no unified interface.

Research Gaps and Future Directions

While systems for data curation, valuation, and drift detection have advanced, autonomous, data-driven machine-learning systems continue to be a difficulty due to unmet research needs. There is still a need for a unified theory of data value that effectively incorporates accuracy and robustness, as well as fairness and drift sensitivity, without resorting to inefficient and time-consuming approaches, such as Shapley values and influence functions. Most research still requires limited automation when it comes to data curation, as it still requires a great deal of Manual Input. Manual Quality Rules need to be established for multilayered, diverse data sets. Consequently, future research will focus on automatic data curation, where minimal human input is required for setting data constraints, anomaly detection, and data repair. The ethical implications of this work must be considered, as insufficient data cleaning and data valuation may further bias and exclude datasets from disadvantaged groups. This shows the importance of fair data curation and transparent valuation systems. Fully autonomous data streams are ideal, as they enable integrated systems to identify and diagnose data drift, acquire relevant data, curate the data, and retrain models in a fully automated manner. Realising this ideal vision will be challenging. It will require shifts and advances in research focused on causal inference, meta-learning, and reinforcement learning. But it will also need to shift the emphasis from developing dependable and trustworthy model-based machine learning systems to a data-centric AI strategy.

Practical Deployment Checklist for Long-Term Model Health

To strengthen the theoretical approaches of this review and the practical implementation of MLOps, this paper presents a useful checklist for real-world applications, detailing the core elements required to maintain the health of models in production for the foreseeable future.

Phase 1: Foundation (Pre-Deployment)

- Specify Data Schemas: Use frameworks such as Great Expectations and TFDV to determine the expected data types, ranges, and constraints for each input feature.
- Set Benchmark Metrics: Compute statistics (mean, variance, quantiles) as a reference point for drift detection for the training data set.

- Data Unit Tests: Consider data as code and write unit tests that examine the data transformation logic and the pipeline integrity.

Phase 2: Monitoring (post-deployment)

- Automate Drift Detection: Utilize drift detectors (K-S test, PSI) for every input feature; set alerts for abnormal changes.
- Monitor Data Health Index: Use a dashboard that visualizes the Data Health Index (completeness, uniqueness, validity) in a real-time manner.
- Active Monitoring of Model Performance: If there are delays in obtaining the ground truth labels, closely analyze proxy metrics (e.g., stability in the prediction distribution).

Phase 3: Adaptation (Continuous Loop)

- Create Feedback Loops: Feed ground truth labels (or human feedback) back into the system.
- Implement Active Learning: To utilise the labelling budget most efficiently, use uncertainty sampling to choose the most valuable production samples for labelling.
- Automate Retraining: Establish a system that automatically initiates model retraining whenever the drift or a decrease in performance surpasses a certain level, incorporating the recently updated data.
- Incorporate Data Valuation: Conduct data valuation (Influence Functions, for example) at regular intervals to detect and eliminate detrimental training instances that may be adversely affecting model performance.

Conclusion

This review has explored data-centric strategies necessary to maintain the long-term health of machine learning models, considering the phenomenon of model erosion, which is influenced by the ageing of AI and shifting data distributions. The key justification is that the data-centric shift in machine learning is not optional but a necessity in the quest to design sustainable AI. Data is no longer a static resource; it requires constant surveillance, curation, and modification during the model's operational lifecycle.

The convergence of drift detection, active learning, data valuation, and data curation creates the ecosystem of adaptable, resilient machine learning systems. The integration of these components into contemporary MLOps systems enables the creation of self-sufficient data pipelines that adapt to changes in the external environment. Achieving this goal requires that data management in ML systems be given a first-class status, and several open problems in coordinated data valuation, fair curation, and full automation be addressed.

It is predicted that the future of ML systems heavily relies on the quality of oversight dominantly exerted on the data ecosystem supporting these systems, as opposed to the architecture of the systems themselves. It is expected that the rapid automation of ML systems will render unmoderated data evolution the key architect of system failure.

Reference

Attenberg, J., & Provost, F. (2011). Selective data acquisition for machine learning. In *Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '11)* (pp. 12–20).

Baena-García, M., del Campo-Ávila, J., Fidalgo, R., Bifet, A., Gavalda, R., & Morales-Bueno, R. (2006). Early drift detection method. In *Fourth International Workshop on Knowledge Discovery from Data Streams (KDDS 2006)*.

Baylor, D., Breck, E., Cheng, H. T., Fiedel, N., Foo, C. Y., Haque, Z., ... & Zinkevich, M. (2017). TFX: A TensorFlow-based production-scale machine learning platform. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 1387–1395).

Bhardwaj, E., Gujral, H., Wu, S., Zogheib, C., Maharaj, T., & Becker, C. (2024). Machine learning data practices through a data curation lens: An evaluation framework. In *The 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT '24)*. https://doi.org/10.1145/3630106.3658955

Bifet, A., & Gavalda, R. (2007). Learning from time-changing data with adaptive windowing. In *Proceedings of the 7th SIAM International Conference on Data Mining (SDM 2007)*.

Breck, E., Cai, S., Nielsen, E., Salib, M., & Sculley, D. (2019). The ML test score: A rubric for ML production readiness and technical debt reduction. In *Proceedings of the 2019 IEEE International Conference on Big Data* (pp. 1123–1132).

Chai, C., Li, G., Li, Y., & Chen, C. (2022). Selective data acquisition in the wild for model charging. *Proceedings of the VLDB Endowment, 15*(5), 1466–1478. https://doi.org/10.14778/3523210.3523223

Evans, N. J., & Liu, Y. (2024). Data valuation with gradient similarity. *arXiv Preprint*, arXiv:2405.08217.

Gama, J., Medas, P., Castillo, G., & Rodrigues, P. (2004). Learning with drift detection. In *Proceedings of the 17th Brazilian Symposium on Artificial Intelligence (SBIA 2004)* (pp. 286–295).

Ghorbani, A., & Zou, J. (2019). Data Shapley: Equitable valuation of data for machine learning. In *Proceedings of the 36th International Conference on Machine Learning (ICML 2019), PMLR* (Vol. 97).

Guha, S., Khan, F. A., & Stoyanovich, J. (2024). Automated data cleaning can compromise fairness in machine learning-based decision-making. *IEEE Transactions on Knowledge and Data Engineering, 36*(1), 51–63. https://doi.org/10.1109/TKDE.2024.3354478

Koh, P. W., & Liang, P. (2017). Understanding black-box predictions via influence functions. In *Proceedings of the 34th International Conference on Machine Learning (ICML 2017), PMLR* (Vol. 70).

Liberty, E., Karnin, Z., Xiang, B., Ruan, L., & Yakhnenko, O. (2020). Elastic machine learning algorithms in Amazon SageMaker. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data* (pp. 731–737).

Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., & Zhang, G. (2018). Learning under concept drift: A review. *IEEE Transactions on Knowledge and Data Engineering, 31*(12), 2346–2363. https://doi.org/10.1109/TKDE.2018.2876857

Mumuni, A., & Mumuni, F. (2024). Automated data processing and feature engineering for deep learning applications: A review. *Engineering Applications of Artificial Intelligence, 131*, 107795. https://doi.org/10.1016/j.engappai.2024.107795

NannyML. (2023, April 11). 91% of ML models degrade in time MIT paper review. https://www.nannyml.com/blog/91-of-ml-perfomance-degrade-in-time

Naser, M. Z. (2026). When machine learning models retire, decay, or become obsolete: The "end-of-life" of AI. *Patterns, 7*(1), 100904. https://doi.org/10.1016/j.patter.2025.100904

Nguyen, T. V., Diakiw, S. M., VerMilyea, M. D., Dinsmore, A. W., & Perreault-Micale, C. L. (2023). Efficient automated error detection in medical data using deep learning and label clustering. *Scientific Reports, 13*(1), 18346. https://doi.org/10.1038/s41598-023-45946-y

Northcutt, C., Jiang, L., & Chuang, I. (2021). Confident learning: Estimating uncertainty in dataset labels. *Journal of Artificial Intelligence Research, 70*, 1373–1411. https://doi.org/10.1613/jair.1.12125

Page, E. S. (1954). Continuous inspection schemes. *Biometrika, 41*(1–2), 100–115.

Raj, A., & Nagi, J. (2022). Convergence of uncertainty sampling for active learning. In *Proceedings of the 39th International Conference on Machine Learning (ICML 2022), PMLR* (Vol. 162).

Schelter, S., Biessmann, F., Januschowski, T., Salinas, D., Seufert, S., & Szarvas, G. (2018). On challenges in machine learning model management. *IEEE Data Engineering Bulletin, 41*(4), 5–15.

Schelter, S., Lange, D., Schmidt, P., Celikel, M., Biessmann, F., & Grafberger, A. (2018). Automating large-scale data quality verification. *Proceedings of the VLDB Endowment, 11*(12), 1781–1794.

Vela, D., Sharp, A., Zhang, R., Nguyen, T., Hoang, A., & Pianykh, O. S. (2022). Temporal quality degradation in AI models. *Scientific Reports, 12*, Article 11654. https://doi.org/10.1038/s41598-022-15245-z